

**Entrega Final: Implementación de Métodos Numéricos
para Análisis y Optimización de Sistemas Universitarios**

Esteban Gómez León
esteban.gomezl@uqvirtual.edu.co

Juan Antonio Betancourt Parra
juana.betancourtp@uqvirtual.edu.co

Docente: Edwin Romero Cuero

Universidad del Quindío
Facultad de Ingeniería
Programa Ingeniería de Sistemas y Computación
Armenia
2026-1



**UNIVERSIDAD
DEL QUINDÍO**
Res. MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN





UNIVERSIDAD
DEL QUINDÍO
Res. MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN



Agradecimientos

Agradecemos al docente Edwin Romero Cuero por el acompañamiento brindado durante el desarrollo de la asignatura de Análisis Numérico. Su forma de orientar la materia permitió comprender la relación entre el cálculo, la estadística y la computación desde una perspectiva práctica y aplicada, mostrando cómo los métodos numéricos pueden convertirse en herramientas útiles para resolver problemas reales mediante procesos iterativos, análisis de datos e implementación computacional.

De manera especial, esta asignatura representó una oportunidad para fortalecer el interés por la programación aplicada a modelos matemáticos, la construcción de algoritmos y la interpretación de resultados numéricos. El desarrollo de este trabajo refleja no solo la aplicación de los contenidos vistos en clase, sino también el entusiasmo generado por una materia que permitió integrar el razonamiento matemático con la lógica computacional.

Índice

Índice.....	4
Índice de Ecuaciones.....	6
1. Introducción.....	7
2. Objetivos.....	7
2.1. Objetivo General.....	7
2.2. Objetivos Específicos.....	7
3. Aproximación por Series de Taylor.....	8
3.1. Descripción del Problema.....	8
3.2. Implementación del Método Numérico.....	9
3.2.1. Modelo Matemático.....	9
3.2.2. Algoritmo Implementado.....	9
3.2.3. Implementación en MATLAB.....	10
3.3. Validación del Resultado.....	13
3.4. Análisis y Conclusiones.....	13
3.5. Interpretación Técnica de Resultados.....	14
4. Métodos de Búsqueda de Raíces de Ecuaciones.....	15
4.1. Descripción del Problema.....	15
4.2. Implementación del Método Numérico.....	17
4.2.1. Modelo Matemático.....	17
4.2.2. Algoritmo Implementado.....	17
4.2.3. Implementación en MATLAB.....	18
4.3. Validación del Resultado.....	25
4.4. Análisis y Conclusiones.....	25
4.5. Interpretación Técnica de Resultados.....	26
5. Interpolación Polinómica para Datos Experimentales.....	27
5.1. Descripción del Problema.....	27
5.2. Implementación del Método Numérico.....	28
5.2.1. Modelo Matemático.....	28
5.2.2. Algoritmo Implementado.....	29
5.2.3. Implementación en MATLAB.....	29
5.3. Validación del Resultado.....	34
5.4. Análisis y Conclusiones.....	35
5.5. Interpretación Técnica de Resultados.....	36
6. Integración Numérica de Datos Irregulares.....	36
6.1. Descripción del Problema.....	36
6.2. Implementación del Método Numérico.....	37
6.2.1. Modelo Matemático.....	37



6.2.2. Algoritmo Implementado.....	38
6.2.3. Implementación en MATLAB.....	38
6.3. Validación del Resultado.....	41
6.4. Análisis y Conclusiones.....	42
6.5. Interpretación Técnica de Resultados.....	43
7. Solución Numérica de EDOs.....	43
7.1. Descripción del Problema.....	43
7.2. Implementación del Método Numérico.....	44
7.2.1. Modelo Matemático.....	44
7.2.2. Algoritmo Implementado.....	45
7.2.3. Implementación en MATLAB.....	45
7.3. Validación del resultado.....	49
7.4. Análisis y conclusiones.....	50
7.5. Interpretación Técnica de Resultados.....	51
8. Conclusiones Generales.....	52
9. Conclusión Final del Documento.....	52
10. Referencias bibliográficas.....	53

Índice de Ecuaciones

Ecuación 1: Fórmula General de las Series de Taylor para una Función de x	9
Ecuación 2: Función Continua que Representa el Porcentaje de CPU Utilizado en Base al Tiempo.....	9
Ecuación 3: Función No Lineal que Modela el Tiempo de Respuesta de un Servidor en Base a Usuarios Conectados.....	15
Ecuación 4: Ecuación del Teorema de Bolzano para Dividir un Intervalo en Dos Partes Iguales.....	17
Ecuación 5: Ecuación del n -ésimo menos uno Término para el Método de Búsqueda de Raíces de Newton-Raphson.....	17
Ecuación 6: Método de Lagrange para Construir Polinomios Interpolantes.....	29
Ecuación 7: Productoria de Sub-términos para hallar el Polinomio Interpolante de Lagrange	29
Ecuación 8: Método de Diferencias Divididas de Newton para hallar el Polinomio Interpolante de Newton.....	29
Ecuación 9: Función Trascendental que Representa el Tráfico de Red en Función al Tiempo en Horas.....	37
Ecuación 10: Método de Trapecio para Aproximar Áreas dada una Función de X	37
Ecuación 11: Método de Simpson para Aproximar Áreas dada una Función de X con Número de Intervalos par.....	37
Ecuación 12: Ecuación Diferencial Ordinaria que Representa un Modelo Logístico de Crecimiento de Usuarios en Función al Tiempo en Horas.....	44
Ecuación 13: Ecuación del Método de Euler Lineal para Encontrar el n -ésimo más uno Término Solución.....	44
Ecuación 14: Ecuación del Método de Runge-Kutta 4 para Encontrar el n -ésimo más uno Término Solución.....	44

1. Introducción

En la actualidad, las plataformas digitales universitarias dependen de servidores capaces de responder de manera eficiente a múltiples solicitudes simultáneas de usuarios. Sistemas como aulas virtuales, portales académicos, inscripción de materias y consultas institucionales presentan variaciones constantes en la demanda, especialmente en horarios de alta concurrencia. Estas condiciones generan la necesidad de analizar el comportamiento del rendimiento computacional y prever escenarios de saturación o degradación del servicio.

Los métodos numéricos constituyen una herramienta fundamental para modelar, aproximar y resolver problemas reales donde no siempre es posible obtener soluciones exactas mediante procedimientos analíticos. A través de técnicas computacionales implementadas en MATLAB, es posible estudiar fenómenos dinámicos asociados al tráfico de red, uso de CPU, tiempos de respuesta y crecimiento de solicitudes en sistemas informáticos.

En este proyecto se plantea el análisis del rendimiento de un servidor web universitario como caso de estudio, utilizando diferentes métodos numéricos que permitan estimar comportamientos futuros, interpolar datos experimentales, calcular acumulaciones de carga y resolver modelos matemáticos asociados al sistema.

2. Objetivos

2.1. Objetivo General

Aplicar e implementar en MATLAB diversos métodos numéricos para analizar y modelar el comportamiento del rendimiento de un servidor web universitario, con el fin de obtener soluciones aproximadas útiles para la toma de decisiones técnicas.

2.2. Objetivos Específicos

- 2.2.1. Modelar el comportamiento del tráfico y carga de un servidor web universitario mediante herramientas matemáticas y computacionales.
- 2.2.2. Implementar en MATLAB métodos numéricos que permitan analizar fenómenos relacionados con el rendimiento del servidor.
- 2.2.3. Utilizar series de Taylor para aproximar funciones asociadas al comportamiento dinámico del sistema.
- 2.2.4. Aplicar métodos de búsqueda de raíces para identificar puntos críticos relacionados con saturación, tiempos de respuesta o estabilidad del servidor.



- 2.2.5. Emplear interpolación polinómica para estimar valores intermedios a partir de datos simulados del rendimiento del sistema.
- 2.2.6. Aplicar técnicas de integración numérica para calcular acumulaciones de carga, tráfico o consumo de recursos en determinados intervalos de tiempo.
- 2.2.7. Resolver ecuaciones diferenciales ordinarias mediante métodos numéricos para simular la evolución temporal del comportamiento del servidor.
- 2.2.8. Analizar los resultados obtenidos en MATLAB para evaluar el desempeño del sistema bajo diferentes escenarios de demanda.
- 2.2.9. Generar conclusiones técnicas que permitan comprender posibles comportamientos futuros y apoyar la toma de decisiones en entornos informáticos académicos.

3. Aproximación por Series de Taylor

3.1. Descripción del Problema

En los sistemas informáticos modernos, el uso del procesador de un servidor puede incrementarse rápidamente durante periodos de alta demanda, como procesos de matrícula académica, consultas simultáneas o acceso masivo a plataformas virtuales. Estas variaciones pueden provocar lentitud en las respuestas del sistema, degradación del servicio o incluso interrupciones temporales si no se detectan oportunamente.

Por esta razón, resulta útil estimar el comportamiento de la carga del CPU en intervalos cortos de tiempo para anticipar posibles sobrecargas y facilitar la toma de decisiones preventivas relacionadas con la administración de recursos computacionales.

Para abordar este problema, se plantea modelar el porcentaje de utilización del procesador mediante una función continua dependiente del tiempo y aproximar su comportamiento local utilizando Series de Taylor. Este método numérico permitirá obtener aproximaciones polinómicas de la función original y analizar la evolución de la carga del sistema cerca de un punto específico.

3.2. Implementación del Método Numérico

3.2.1. Modelo Matemático

La aproximación mediante Series de Taylor permite representar una función alrededor de un punto a mediante un polinomio de grado n .

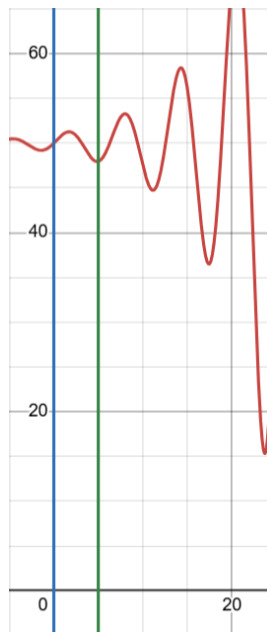
$$f(x) \approx f(a) + f'(a)(x - a) + \frac{f''(a)}{2!}(x - a)^2 + \dots + \frac{f^n(a)}{n!}(x - a)^n$$

Ecuación 1: Fórmula General de las Series de Taylor para una Función de x

Para este proyecto, se modeló el porcentaje de uso del CPU mediante una función continua dependiendo del tiempo:

$$f(t) \approx e^{0.15t} \sin(t) + 50$$

Ecuación 2: Función Continua que Representa el Porcentaje de CPU Utilizado en Base al Tiempo



La aproximación se realizó alrededor de $t = 0$ utilizando un polinomio de Taylor de orden 2, 4 y 6 en un intervalo de análisis de $0 \leq t \leq 5$

3.2.2. Algoritmo Implementado

- 3.2.2.1. Definir la función original y sus derivadas
- 3.2.2.2. Seleccionar el punto de expansión
- 3.2.2.3. Calcular los coeficientes del polinomio de Taylor
- 3.2.2.4. Evaluar el polinomio en los puntos deseados
- 3.2.2.5. Comparar la aproximación con la función original



3.2.3. Implementación en MATLAB

```
function [approximatedValues, coefficients, metadata] =
    taylor_approximation( ...
        functionHandle, derivativeHandles, expansionPoint,
        evaluationPoints, order)
    narginchk(5, 5);

    validateattributes(expansionPoint, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename,
    'expansionPoint', 3);
    validateattributes(evaluationPoints, {'numeric'}, ...
        {'real', 'finite'}, mfilename, 'evaluationPoints', 4);
    validateattributes(order, {'numeric'}, ...
        {'real', 'finite', 'scalar', 'integer', 'nonnegative'},
    ...
        mfilename, 'order', 5);

    if ~isa(functionHandle, 'function_handle')
        error('taylor_approximation:InvalidFunctionHandle', ...
            'functionHandle must be a valid function handle.');
```

end

```
    if ~iscell(derivativeHandles)
        error('taylor_approximation:InvalidDerivativeContainer',
    ...
            'derivativeHandles must be a cell array of function
handles.');
```

end

```
    if numel(derivativeHandles) < order
        error('taylor_approximation:InsufficientDerivatives', ...
            ['At least %d derivative handles are required for
order %d. ', ...
            'Received %d.'], order, order,
    numel(derivativeHandles));
    end
```



```

    for derivativeIndex = 1:order
        if ~isa(derivativeHandles{derivativeIndex},
            'function_handle')
            error('taylor_approximation:InvalidDerivativeHandle',
                ...
                'Each derivative entry must be a function
handle. ');
        end
    end

    originalSize = size(evaluationPoints);
    flattenedEvaluationPoints = evaluationPoints(:);

    coefficients = zeros(1, order + 1);
    termValuesAtExpansionPoint = zeros(1, order + 1);

    baseValue = functionHandle(expansionPoint);
    validateattributes(baseValue, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename,
        'functionHandle(expansionPoint)');

    coefficients(1) = baseValue;
    termValuesAtExpansionPoint(1) = baseValue;

    for derivativeOrder = 1:order
        derivativeValue =
            derivativeHandles{derivativeOrder}(expansionPoint);

        validateattributes(derivativeValue, {'numeric'}, ...
            {'real', 'finite', 'scalar'}, mfilename, ...
            sprintf('derivativeHandles{%d}(expansionPoint)',
                derivativeOrder));

        termValuesAtExpansionPoint(derivativeOrder + 1) =
            derivativeValue;
        coefficients(derivativeOrder + 1) = derivativeValue /

```



```
factorial(derivativeOrder);
end

shiftedPoints = flattenedEvaluationPoints - expansionPoint;
approximatedValues = zeros(size(flattenedEvaluationPoints));

for derivativeOrder = 0:order
    approximatedValues = approximatedValues + ...
        coefficients(derivativeOrder + 1) .* shiftedPoints .^
derivativeOrder;
end

approximatedValues = reshape(approximatedValues,
originalSize);

metadata = struct( ...
    'expansionPoint', expansionPoint, ...
    'order', order, ...
    'polynomialHandle', @(x)
evaluateTaylorPolynomial(coefficients, expansionPoint, x), ...
    'termValuesAtExpansionPoint',
termValuesAtExpansionPoint);
end

function values = evaluateTaylorPolynomial(coefficients,
expansionPoint, evaluationPoints)
    shiftedPoints = evaluationPoints - expansionPoint;
    values = zeros(size(evaluationPoints));

    for derivativeOrder = 0:(numel(coefficients) - 1)
        values = values + coefficients(derivativeOrder + 1) .*
shiftedPoints .^ derivativeOrder;
    end
end
```

3.3. Validación del Resultado

La validación de la aproximación se realizó comparando los valores obtenidos mediante los polinomios de Taylor de orden 2, 4 y 6 con los valores de la función original que modela el comportamiento de la carga del CPU.

Se utilizaron representaciones gráficas y el cálculo del error absoluto para evaluar la precisión de cada aproximación dentro del intervalo de análisis $0 \leq t \leq 5$

Las gráficas generadas en MATLAB permitieron observar que:

- El polinomio de segundo orden presenta una aproximación aceptable únicamente cerca del punto de expansión.
- Los polinomios de cuarto y sexto orden ofrecen un ajuste significativamente más preciso.
- El error aumenta progresivamente a medida que los valores de t se alejan del punto $t = 0$.

Además, el archivo *summary_report.txt* almacena los resultados numéricos obtenidos durante la simulación, incluyendo coeficientes del polinomio y errores calculados para cada aproximación.

3.4. Análisis y Conclusiones

La implementación de las Series de Taylor permitió aproximar correctamente el comportamiento local de la función que representa la carga del CPU del servidor.

Los resultados evidenciaron que:

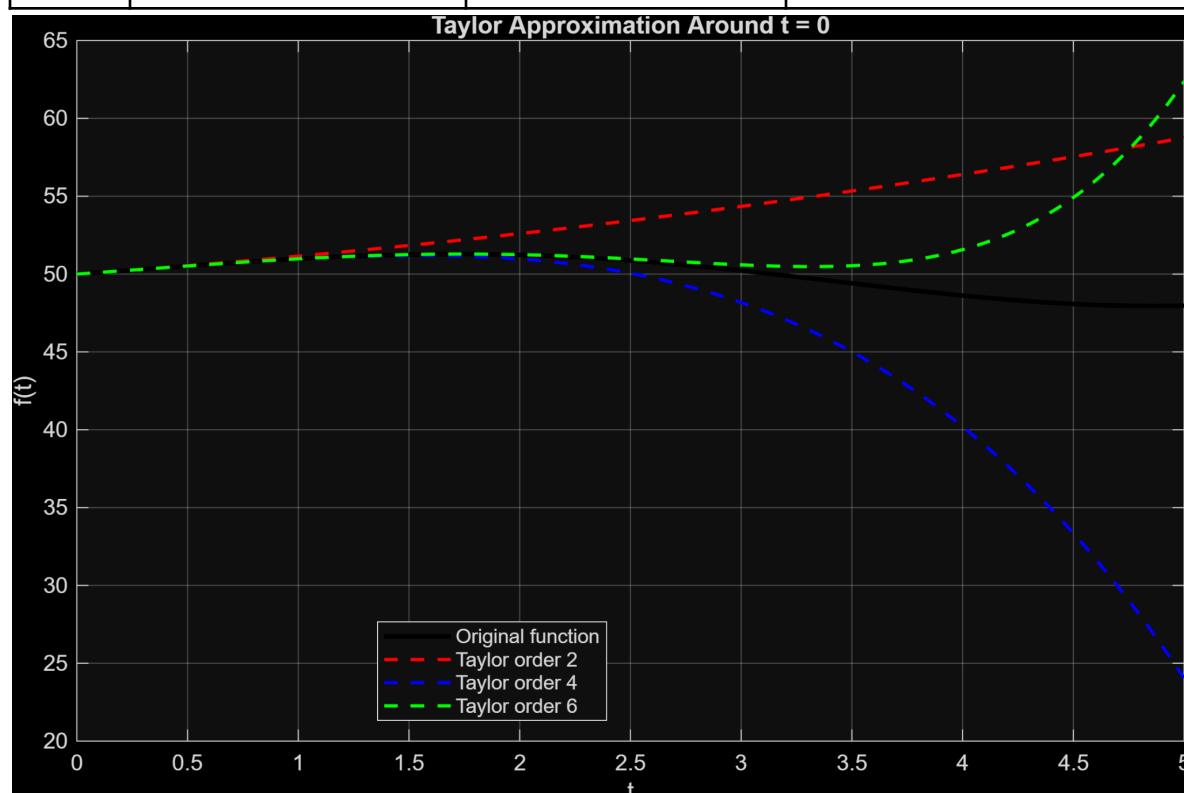
- El aumento en el orden del polinomio mejora la precisión de la aproximación.
- Las aproximaciones de bajo orden presentan desviaciones mayores fuera del entorno cercano al punto de expansión.
- MATLAB facilitó la automatización de cálculos y la representación gráfica de los resultados.

Desde el punto de vista computacional, el método resulta útil para simplificar funciones complejas y realizar estimaciones rápidas del comportamiento del sistema.

Finalmente, se concluye que las Series de Taylor constituyen una herramienta eficiente para el análisis numérico de variables dinámicas en entornos de ingeniería de sistemas, especialmente cuando se requiere estudiar el comportamiento local de una función de manera aproximada.



Orden	Error Absoluto Máximo	Error Absoluto Medio	Error Absoluto Máximo Cerca de la Expansión
2	10.7800427053922	14.404025452788	1.35775464656391
4	23.9204781279412	4.199543556545	0.25676215462574
6	14.404025452788	1.79060830370181	0.0190973374699936



3.5. Interpretación Técnica de Resultados

Los resultados confirman que las Series de Taylor son efectivas principalmente en el entorno cercano al punto de expansión. El polinomio de orden 6 redujo el error cercano a la expansión de 1.3577 a 0.0190 frente al orden 2, lo que representa una mejora aproximada del 98.6 %. Sin embargo, el error máximo global no disminuyó de manera estrictamente monótonica, lo que evidencia que aumentar el orden no garantiza una mejor aproximación en todo el intervalo cuando el punto de expansión se mantiene fijo en $t = 0$. Por tanto, para estimar el uso de CPU en intervalos más alejados sería recomendable utilizar expansiones locales por tramos o seleccionar un punto de expansión más cercano a la zona de interés.

4. Métodos de Búsqueda de Raíces de Ecuaciones

4.1. Descripción del Problema

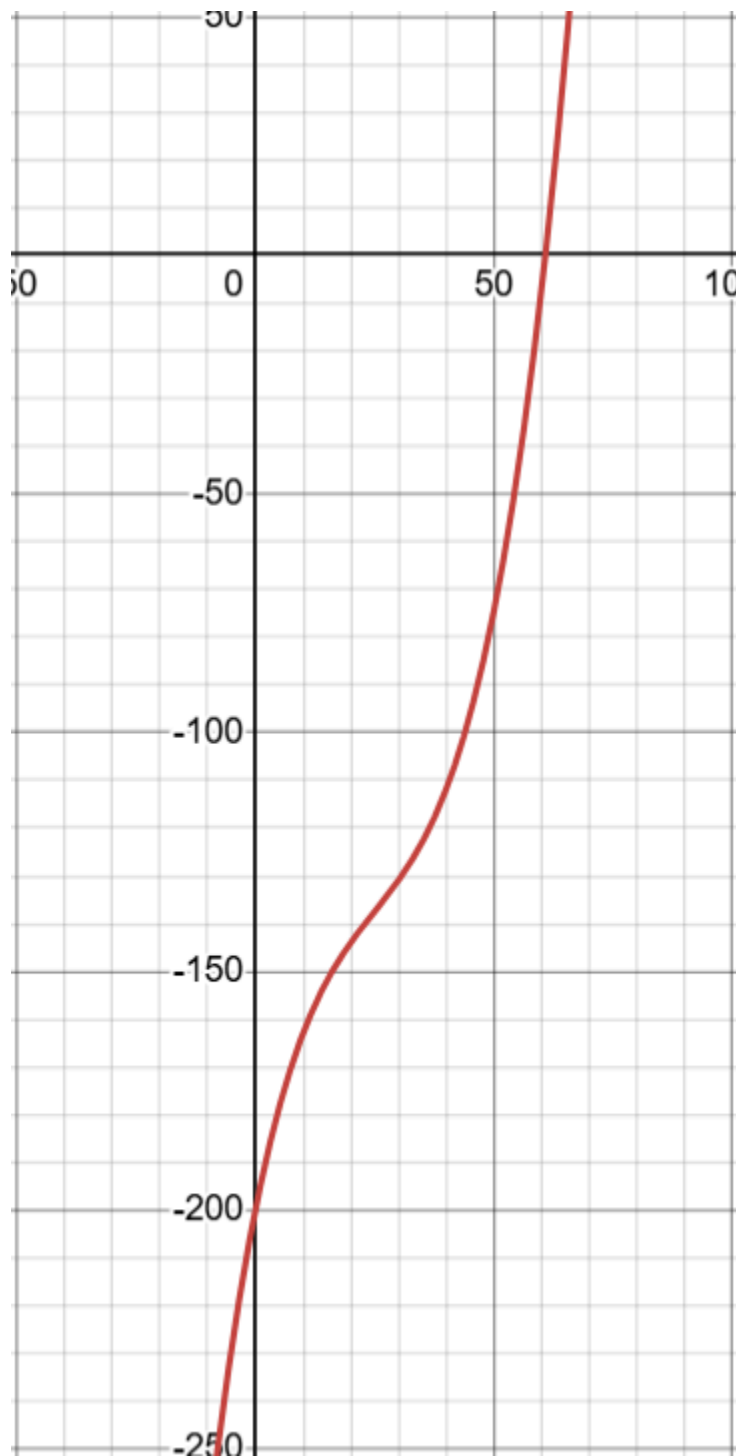
En plataformas digitales universitarias, el tiempo de respuesta de un servidor depende directamente de la cantidad de usuarios conectados simultáneamente. A medida que aumenta la demanda, el sistema puede acercarse a niveles críticos donde el tiempo de respuesta supera los valores aceptables para una operación eficiente.

Determinar el número aproximado de usuarios simultáneos para el cual el tiempo de respuesta alcanza el límite máximo permitido resulta importante para anticipar escenarios de congestión y tomar decisiones relacionadas con escalabilidad y optimización del sistema.

Para abordar este problema, se plantea una ecuación no lineal que modela el comportamiento del tiempo de respuesta del servidor en función del número de usuarios conectados:

$$f(x) = 0.002x^3 - 0.15x^2 + 5x - 200$$

Ecuación 3: Función No Lineal que Modela el Tiempo de Respuesta de un Servidor en Base a Usuarios Conectados



donde:

- x representa la cantidad de usuarios simultáneos.
- $f(x) = 0$ representa el punto crítico del sistema.

La raíz de la ecuación corresponde a la cantidad de usuarios para la cual el servidor alcanza el límite operacional definido

Para resolver este problema se emplearán los métodos numéricos de Bisección y Newton-Raphson, permitiendo comparar precisión, estabilidad y velocidad de convergencia

4.2. Implementación del Método Numérico

4.2.1. Modelo Matemático

Los métodos de búsqueda de raíces permiten encontrar soluciones aproximadas de ecuaciones no lineales mediante procesos iterativos.

El método de Bisección se basa en dividir repetidamente un intervalo donde existe cambio de signo, basándose en el **Teorema de Bolzano**:

$$c = \frac{a+b}{2}$$

Ecuación 4: Ecuación del Teorema de Bolzano para Dividir un Intervalo en Dos Partes Iguales

Por otra parte, el método de Newton-Raphson utiliza derivadas para aproximar rápidamente la raíz:

$$x_{n-1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Ecuación 5: Ecuación del n-ésimo menos uno Término para el Método de Búsqueda de Raíces de Newton-Raphson

Para este proyecto se utilizaron los siguiente parámetros:

Método de Bisección

- Intervalo inicial: [20, 80]
- Tolerancia: 10^{-6}
- Máximo de iteraciones: 100

Método de Newton-Raphson

- Valor inicial: $x_n = 50$
- Tolerancia: 10^{-6}
- Máximo de iteraciones: 100

4.2.2. Algoritmo Implementado

4.2.2.1. Método de Bisección

- Definir el intervalo inicial.
- Evaluar el cambio de signo de la función.
- Calcular el punto medio.
- Seleccionar el nuevo intervalo.
- Repetir hasta cumplir la tolerancia.

4.2.2.2. Método de Newton-Raphson

- Definir una aproximación inicial.
- Evaluar la función y su derivada.
- Calcular la siguiente aproximación.
- Evaluar el error.
- Repetir hasta convergencia.

4.2.3. Implementación en MATLAB

4.2.3.1. Método de Bisección

```
function [root, iterationCount, metadata] = bisection_method( ...
    functionHandle, leftEndpoint, rightEndpoint, tolerance,
    maxIterations)
    narginchk(5, 5);

    if ~isa(functionHandle, 'function_handle')
        error('bisection_method:InvalidFunctionHandle', ...
            'functionHandle must be a valid function handle.');
```

end

```
    validateattributes(leftEndpoint, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename, 'leftEndpoint',
    2);
    validateattributes(rightEndpoint, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename, 'rightEndpoint',
    3);
    validateattributes(tolerance, {'numeric'}, ...
        {'real', 'finite', 'scalar', 'positive'}, mfilename,
    'tolerance', 4);
    validateattributes(maxIterations, {'numeric'}, ...
        {'real', 'finite', 'scalar', 'integer', 'positive'}, ...
        mfilename, 'maxIterations', 5);

    if leftEndpoint >= rightEndpoint
        error('bisection_method:InvalidInterval', ...
            'leftEndpoint must be smaller than rightEndpoint.');
```

end



```

leftValue = functionHandle(leftEndpoint);
rightValue = functionHandle(rightEndpoint);

validateattributes(leftValue, {'numeric'}, ...
    {'real', 'finite', 'scalar'}, mfilename,
'functionHandle(leftEndpoint)');
validateattributes(rightValue, {'numeric'}, ...
    {'real', 'finite', 'scalar'}, mfilename,
'functionHandle(rightEndpoint)');

if leftValue == 0
    root = leftEndpoint;
    iterationCount = 0;
    metadata = buildMetadata(true, leftEndpoint,
rightEndpoint, tolerance, ...
        maxIterations, leftValue, rightValue, root, 0,
abs(leftValue), true);
    return;
end

if rightValue == 0
    root = rightEndpoint;
    iterationCount = 0;
    metadata = buildMetadata(true, leftEndpoint,
rightEndpoint, tolerance, ...
        maxIterations, leftValue, rightValue, root, 0,
abs(rightValue), true);
    return;
end

if leftValue * rightValue > 0
    error('bisection_method:NoSignChange', ...
        'The interval must contain a sign change.');
```

```

end

intervalHistory = zeros(maxIterations, 2);
```



```

midpointHistory = zeros(maxIterations, 1);
functionHistory = zeros(maxIterations, 1);
errorHistory = zeros(maxIterations, 1);

root = NaN;
converged = false;

for iterationIndex = 1:maxIterations
    midpoint = (leftEndpoint + rightEndpoint) / 2;
    midpointValue = functionHandle(midpoint);

    validateattributes(midpointValue, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename,
        'functionHandle(midpoint)');

    intervalHistory(iterationIndex, :) = [leftEndpoint,
rightEndpoint];
    midpointHistory(iterationIndex) = midpoint;
    functionHistory(iterationIndex) = midpointValue;
    errorHistory(iterationIndex) = (rightEndpoint -
leftEndpoint) / 2;

    root = midpoint;

    if abs(midpointValue) <= tolerance ||
errorHistory(iterationIndex) <= tolerance
        iterationCount = iterationIndex;
        converged = true;
        break;
    end

    if leftValue * midpointValue < 0
        rightEndpoint = midpoint;
        rightValue = midpointValue;
    else
        leftEndpoint = midpoint;

```



```

        leftValue = midpointValue;
    end
end

if ~converged
    iterationCount = maxIterations;
end

metadata = buildMetadata(converged, leftEndpoint,
rightEndpoint, tolerance, ...
    maxIterations, leftValue, rightValue, root,
iterationCount, ...
    abs(functionHistory(iterationCount)), false);
metadata.intervalHistory = intervalHistory(1:iterationCount,
:);
metadata.midpointHistory = midpointHistory(1:iterationCount);
metadata.functionHistory = functionHistory(1:iterationCount);
metadata.errorHistory = errorHistory(1:iterationCount);
end

function metadata = buildMetadata(converged, leftEndpoint,
rightEndpoint, tolerance, ...
    maxIterations, leftValue, rightValue, root, iterationCount,
finalResidual, hitEndpoint)
    metadata = struct( ...
        'converged', converged, ...
        'leftEndpoint', leftEndpoint, ...
        'rightEndpoint', rightEndpoint, ...
        'tolerance', tolerance, ...
        'maxIterations', maxIterations, ...
        'leftValue', leftValue, ...
        'rightValue', rightValue, ...
        'root', root, ...
        'iterationCount', iterationCount, ...
        'finalResidual', finalResidual, ...
        'hitEndpoint', hitEndpoint);
end

```



end

4.2.3.2. Método de Newton-Raphson

```
function [root, iterationCount, metadata] =
newton_raphson_method( ...
    functionHandle, derivativeHandle, initialGuess, tolerance,
maxIterations)
    narginchk(5, 5);

    if ~isa(functionHandle, 'function_handle')
        error('newton_raphson_method:InvalidFunctionHandle', ...
            'functionHandle must be a valid function handle.');
```

end

```
    if ~isa(derivativeHandle, 'function_handle')
        error('newton_raphson_method:InvalidDerivativeHandle',
...
            'derivativeHandle must be a valid function handle.');
```

end

```
    validateattributes(initialGuess, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename, 'initialGuess',
3);
    validateattributes(tolerance, {'numeric'}, ...
        {'real', 'finite', 'scalar', 'positive'}, mfilename,
'tolerance', 4);
    validateattributes(maxIterations, {'numeric'}, ...
        {'real', 'finite', 'scalar', 'integer', 'positive'}, ...
        mfilename, 'maxIterations', 5);

    approximationHistory = zeros(maxIterations + 1, 1);
    functionHistory = zeros(maxIterations + 1, 1);
    derivativeHistory = zeros(maxIterations + 1, 1);
    relativeStepHistory = zeros(maxIterations, 1);

    currentApproximation = initialGuess;
```




```

approximationHistory(1) = currentApproximation;
converged = false;

for iterationIndex = 1:maxIterations
    currentValue = functionHandle(currentApproximation);
    currentDerivative =
derivativeHandle(currentApproximation);

    validateattributes(currentValue, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename, ...
        'functionHandle(currentApproximation)');
    validateattributes(currentDerivative, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename, ...
        'derivativeHandle(currentApproximation)');

    functionHistory(iterationIndex) = currentValue;
    derivativeHistory(iterationIndex) = currentDerivative;

    if abs(currentDerivative) <= eps
        error('newton_raphson_method:ZeroDerivative', ...
            'The derivative became zero or numerically
unstable.');
```

end

```

        nextApproximation = currentApproximation - currentValue /
currentDerivative;
        approximationHistory(iterationIndex + 1) =
nextApproximation;

        denominator = max(1, abs(nextApproximation));
        relativeStepHistory(iterationIndex) =
abs(nextApproximation - currentApproximation) / denominator;

        if abs(functionHandle(nextApproximation)) <= tolerance ||
...
            relativeStepHistory(iterationIndex) <= tolerance

```



```

        currentApproximation = nextApproximation;
        converged = true;
        iterationCount = iterationIndex;
        break;
    end

    currentApproximation = nextApproximation;
end

if ~converged
    iterationCount = maxIterations;
end

root = currentApproximation;
finalValue = functionHandle(root);

approximationHistory = approximationHistory(1:(iterationCount
+ 1));
functionHistory = functionHistory(1:iterationCount);
derivativeHistory = derivativeHistory(1:iterationCount);
relativeStepHistory = relativeStepHistory(1:iterationCount);

metadata = struct( ...
    'converged', converged, ...
    'initialGuess', initialGuess, ...
    'tolerance', tolerance, ...
    'maxIterations', maxIterations, ...
    'root', root, ...
    'finalResidual', abs(finalValue), ...
    'approximationHistory', approximationHistory, ...
    'functionHistory', functionHistory, ...
    'derivativeHistory', derivativeHistory, ...
    'relativeStepHistory', relativeStepHistory);
end

```

4.3. Validación del Resultado

La validación se realizó comparando las raíces obtenidas mediante los métodos de Bisección y Newton-Raphson.

Se analizaron:

- precisión de la raíz encontrada,
- número de iteraciones requeridas,
- comportamiento de convergencia,
- y tiempo computacional.

Las gráficas y resultados numéricos obtenidos permitieron observar que:

- El método de Newton-Raphson converge más rápidamente.
- El método de Bisección presenta mayor estabilidad.
- Ambos métodos convergen hacia una raíz similar dentro de la tolerancia establecida.

Además, el archivo *summary_report.txt* almacena información relacionada con iteraciones, errores y aproximaciones obtenidas durante la simulación.

4.4. Análisis y Conclusiones

La implementación de los métodos de búsqueda de raíces permitió identificar correctamente el punto crítico asociado al comportamiento del servidor bajo alta demanda.

Los resultados evidenciaron que:

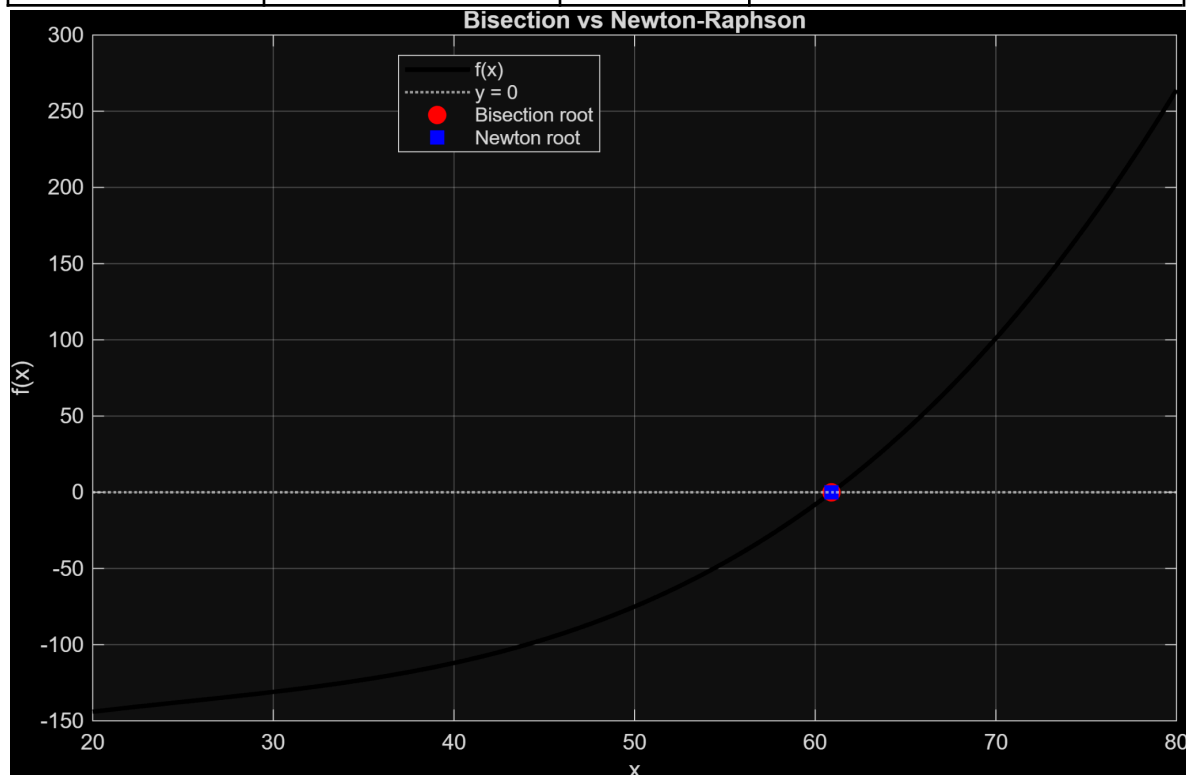
- Newton-Raphson requiere menos iteraciones para alcanzar la convergencia.
- El método de Bisección ofrece mayor robustez ante aproximaciones iniciales poco precisas.
- Ambos métodos son adecuados para resolver problemas no lineales relacionados con rendimiento computacional.

Desde el punto de vista computacional, estos métodos permiten estimar límites operacionales y detectar posibles escenarios de saturación en sistemas informáticos.

Finalmente, se concluye que los métodos de búsqueda de raíces constituyen herramientas fundamentales para el análisis y optimización de sistemas dinámicos en ingeniería de software y administración de servidores.



Método	Raíz	Iteraciones	Residuo Final
Bisección	60.909843146801	26	$1.51884393062574 * 10^{-6}$
Newton-Raphson	60.9098433158037	5	0



4.5. Interpretación Técnica de Resultados

Ambos métodos identificaron un punto crítico cercano a **60.91** usuarios simultáneos, lo que sugiere que, bajo el modelo planteado, el servidor empezaría a alcanzar su límite operacional alrededor de los **61** usuarios concurrentes. Newton-Raphson fue más eficiente, al requerir solo **5** iteraciones frente a las **26** de Bisección, lo que representa una reducción aproximada del **80.8%** en el número de iteraciones. No obstante, Bisección ofrece mayor confiabilidad cuando solo se conoce un intervalo con cambio de signo, mientras que Newton-Raphson depende de la elección del valor inicial y de la estabilidad de la derivada. En un sistema real, Newton sería preferible para monitoreo rápido si se cuenta con buen modelo previo; Bisección sería más seguro para validaciones iniciales.



5. Interpolación Polinómica para Datos Experimentales

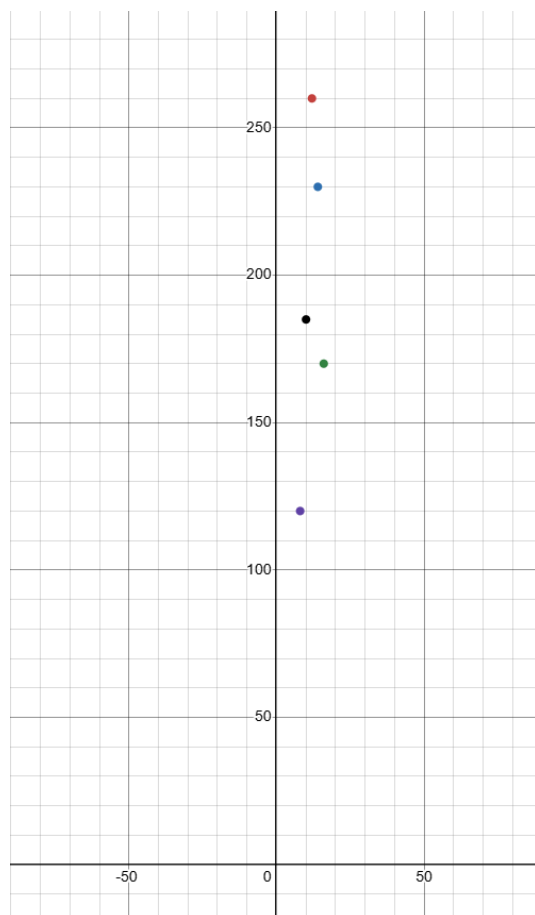
5.1. Descripción del Problema

En los sistemas de monitoreo de redes y servidores es común que algunos registros de tráfico presentan datos faltantes debido a fallos de sensores, interrupciones temporales o pérdidas en la captura de información. La ausencia de estos valores dificulta el análisis histórico del comportamiento del sistema y la toma de decisiones basada en datos.

Para abordar este problema, se plantea estimar valores desconocidos del tráfico de red a partir de mediciones disponibles en distintos instantes de tiempo mediante técnicas de interpolación polinómica.

Se consideran las siguientes mediciones del tráfico promedio de red en un servidor universitario durante una jornada académica:

Tiempo (Horas)	Tráfico (Mbps)
8	120
10	185
12	260
14	230
16	170



Con estos datos se busca estimar el tráfico de red a las 11:00 a.m. y a la 1:00 p.m. ($x = 11, 13$), donde se dispone de registros directos.

Para resolver este problema se emplearán los métodos de interpolación polinómica de Lagrange y Newton, permitiendo comparar precisión, estabilidad y facilidad computacional.

5.2. Implementación del Método Numérico

5.2.1. Modelo Matemático

La interpolación polinómica permite construir una función aproximada que pasa exactamente por un conjunto de puntos conocidos, facilitando la estimación de valores intermedios no registrados.

El método de Lagrange construye el polinomio mediante funciones base:

$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

Ecuación 6: Método de Lagrange para Construir Polinomios Interpolantes

donde:

$$L_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

Ecuación 7: Productoria de Sub-términos para hallar el Polinomio Interpolante de Lagrange

Por otra parte, el método de Newton utiliza diferencias divididas para construir el polinomio de forma incremental:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n \prod_{k=0}^{n-1} (x - x_k)$$

Ecuación 8: Método de Diferencias Divididas de Newton para hallar el Polinomio Interpolante de Newton

Para este proyecto se utilizaron los siguiente parámetros:

- Número de puntos conocidos: 5
- Variable independiente: tiempo (horas)
- Variable dependiente: tráfico de red (Mbps)
- Valores a estimar: $x = 11$ y $x = 13$

5.2.2. Algoritmo Implementado

Método de Lagrange

- Definir los puntos conocidos.
- Construir los polinomios base de Lagrange.
- Calcular el polinomio interpolante.
- Evaluar el polinomio en los puntos deseados.

Método de Newton

- Definir los puntos conocidos.
- Construir la tabla de diferencias divididas.
- Generar el polinomio interpolante.
- Evaluar el polinomio en los puntos requeridos.

5.2.3. Implementación en MATLAB

5.2.3.1. Método de Lagrange

```
function [interpolatedValues, coefficients, metadata] =  
lagrange_interpolation( ...  
    xData, yData, evaluationPoints)  
    narginchk(3, 3);  
    [xData, yData] = validateInterpolationInput(xData, yData,  
    mfilename);
```




```
originalSize = size(evaluationPoints);
evaluationPoints = evaluationPoints(:);

nodeCount = numel(xData);
interpolatedValues = zeros(size(evaluationPoints));
basisDenominators = ones(nodeCount, 1);

vandermondeMatrix = zeros(nodeCount, nodeCount);
for rowIndex = 1:nodeCount
    vandermondeMatrix(rowIndex, :) = xData(rowIndex) .^
(0:(nodeCount - 1));
end
coefficients = (vandermondeMatrix \ yData).';

for basisIndex = 1:nodeCount
    basisPolynomial = ones(size(evaluationPoints));
    denominator = 1;

    for nodeIndex = 1:nodeCount
        if nodeIndex ~= basisIndex
            basisPolynomial = basisPolynomial .* ...
                (evaluationPoints - xData(nodeIndex)) ./ ...
                (xData(basisIndex) - xData(nodeIndex));
            denominator = denominator * (xData(basisIndex) -
xData(nodeIndex));
        end
    end

    interpolatedValues = interpolatedValues +
yData(basisIndex) .* basisPolynomial;
    basisDenominators(basisIndex) = denominator;
end

interpolatedValues = reshape(interpolatedValues,
originalSize);
```



```

metadata = struct( ...
    'nodeCount', nodeCount, ...
    'xData', xData, ...
    'yData', yData, ...
    'basisDenominators', basisDenominators, ...
    'polynomialHandle', @(x) polyval(fliplr(coefficients),
x));
end

function [xData, yData] = validateInterpolationInput(xData,
yData, functionName)
    validateattributes(xData, {'numeric'}, ...
        {'real', 'finite', 'vector', 'nonempty'}, functionName,
'xData', 1);
    validateattributes(yData, {'numeric'}, ...
        {'real', 'finite', 'vector', 'nonempty'}, functionName,
'yData', 2);

    xData = xData(:);
    yData = yData(:);

    if numel(xData) ~= numel(yData)
        error('%s:DimensionMismatch', functionName, ...
            'xData and yData must have the same number of
elements.');
```

```
end
```

```

    if numel(unique(xData)) ~= numel(xData)
        error('%s:RepeatedNodes', functionName, ...
            'xData must not contain repeated interpolation
nodes.');
```

```
end
```

```
end
```

5.2.3.2. Metodo de Newton



```
function [interpolatedValues, dividedDifferenceTable, metadata] =
newton_interpolation( ...
    xData, yData, evaluationPoints)
    narginchk(3, 3);
    [xData, yData] = validateInterpolationInput(xData, yData,
mfilename);

    originalSize = size(evaluationPoints);
    evaluationPoints = evaluationPoints(:);

    nodeCount = numel(xData);
    dividedDifferenceTable = zeros(nodeCount, nodeCount);
    dividedDifferenceTable(:, 1) = yData;

    for columnIndex = 2:nodeCount
        for rowIndex = 1:(nodeCount - columnIndex + 1)
            numerator = dividedDifferenceTable(rowIndex + 1,
columnIndex - 1) - ...
                dividedDifferenceTable(rowIndex, columnIndex -
1);
            denominator = xData(rowIndex + columnIndex - 1) -
xData(rowIndex);
            dividedDifferenceTable(rowIndex, columnIndex) =
numerator / denominator;
        end
    end

    newtonCoefficients = dividedDifferenceTable(1, :);
    interpolatedValues = zeros(size(evaluationPoints));

    for pointIndex = 1:numel(evaluationPoints)
        currentPoint = evaluationPoints(pointIndex);
        currentValue = newtonCoefficients(1);
        cumulativeProduct = 1;

        for coefficientIndex = 2:nodeCount
```



```

        cumulativeProduct = cumulativeProduct * ...
            (currentPoint - xData(coefficientIndex - 1));
        currentValue = currentValue + ...
            newtonCoefficients(coefficientIndex) *
cumulativeProduct;
    end

    interpolatedValues(pointIndex) = currentValue;
end

interpolatedValues = reshape(interpolatedValues,
originalSize);

metadata = struct( ...
    'nodeCount', nodeCount, ...
    'xData', xData, ...
    'yData', yData, ...
    'newtonCoefficients', newtonCoefficients, ...
    'polynomialHandle', @(x)
evaluateNewtonPolynomial(newtonCoefficients, xData, x));
end

function values = evaluateNewtonPolynomial(newtonCoefficients,
xData, evaluationPoints)
    values = zeros(size(evaluationPoints));

    for pointIndex = 1:numel(evaluationPoints)
        currentPoint = evaluationPoints(pointIndex);
        currentValue = newtonCoefficients(1);
        cumulativeProduct = 1;

        for coefficientIndex = 2:numel(newtonCoefficients)
            cumulativeProduct = cumulativeProduct * ...
                (currentPoint - xData(coefficientIndex - 1));
            currentValue = currentValue + ...
                newtonCoefficients(coefficientIndex) *

```



```

cumulativeProduct;
    end

    values(pointIndex) = currentValue;
end
end

function [xData, yData] = validateInterpolationInput(xData,
yData, functionName)
    validateattributes(xData, {'numeric'}, ...
        {'real', 'finite', 'vector', 'nonempty'}, functionName,
        'xData', 1);
    validateattributes(yData, {'numeric'}, ...
        {'real', 'finite', 'vector', 'nonempty'}, functionName,
        'yData', 2);

    xData = xData(:);
    yData = yData(:);

    if numel(xData) ~= numel(yData)
        error('%s:DimensionMismatch', functionName, ...
            'xData and yData must have the same number of
elements.');
```

5.3. Validación del Resultado

La validación se realizó comparando los valores estimados mediante los métodos de Lagrange y Newton para los horarios sin medición directa.

Se analizaron:

- Coincidencia de resultados entre ambos métodos
- Estabilidad numérica
- Complejidad computacional
- Comportamiento gráfico del polinomio interpolante

Las gráficas generadas en MATLAB permitieron observar que:

- Ambos métodos producen resultados equivalentes para los puntos interpolados.
- El comportamiento del polinomio sigue adecuadamente la tendencia de los datos experimentales.
- El tráfico de red presenta crecimiento hacia el mediodía y disminución durante la tarde.

Además, el archivo *summary_report.txt* almacena los valores interpolados, coeficientes y resultados obtenidos durante las simulaciones.

5.4. Análisis y Conclusiones

La implementación de los métodos de interpolación polinómica permitió estimar correctamente valores faltantes del tráfico de red a partir de datos experimentales conocidos.

Los resultados evidenciaron que:

- Los métodos de Lagrange y Newton generan estimaciones equivalentes.
- El método de Newton facilita la incorporación de nuevos datos sin reconstruir completamente el polinomio.
- MATLAB permitió automatizar el cálculo y representación gráfica de los resultados obtenidos.

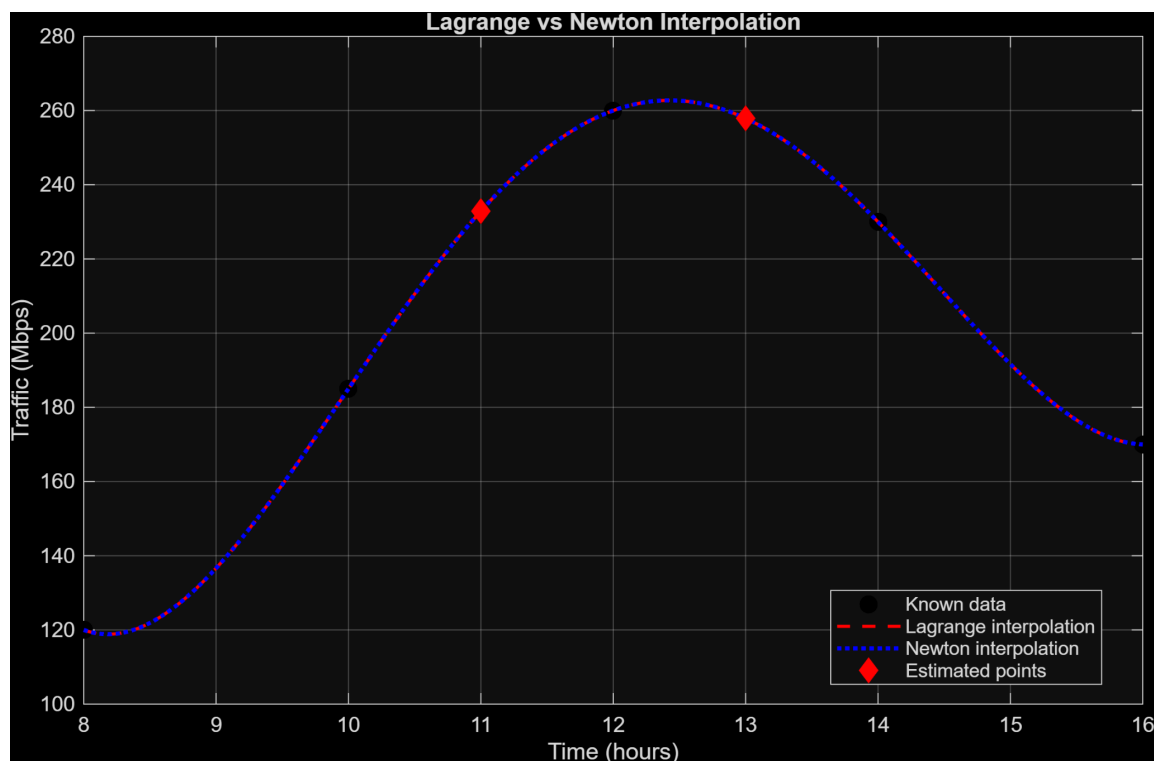
Desde el punto de vista computacional, la interpolación polinómica constituye una herramienta útil para reconstruir información faltante en sistemas de monitoreo y análisis de redes.

Finalmente, se concluye que los métodos de interpolación permiten mejorar el análisis de datos incompletos en entornos computacionales, facilitando la toma de decisiones basada en información aproximada y consistente.

Punto de Evaluación	Valor de Lagrange	Valor de Newton	Diferencia
11	232.890625	232.890625	0



13	257.890625	257.890625	0
----	------------	------------	---



5.5. Interpretación Técnica de Resultados

Los valores interpolados muestran que el tráfico no solo aumenta hacia el mediodía, sino que se mantiene en un nivel alto hasta la 1:00 p.m. A las 11:00 a.m. Se estimó un tráfico de **232.89 Mbps**, mientras que a la 1:00 p.m. se estimó **257.89 Mbps**, valor muy cercano al dato máximo conocido de **260 Mbps** registrado a las 12:00 m. Esto sugiere que la ventana crítica del sistema no ocurre únicamente al mediodía, sino entre las 12:00 m. y la 1:00 p.m. Desde una perspectiva operativa, ese intervalo debería considerarse prioritario para asignar mayor ancho de banda, balanceo de carga o monitoreo preventivo.

6. Integración Numérica de Datos Irregulares

6.1. Descripción del Problema

En la administración de infraestructuras tecnológicas resulta importante conocer la cantidad total de datos transferidos por una red durante un periodo determinado. Aunque pueden obtenerse mediciones instantáneas del tráfico, no siempre se dispone de una función integrable exacta ni de datos continuos en todo momento.



Por esta razón, es necesario emplear métodos numéricos que permitan aproximar el volumen total de información transmitida a partir de mediciones discretas del ancho de banda utilizado.

Para abordar este problema, se propone modelar el tráfico de red mediante la siguiente función continua:

$$f(t) = 40 + 15 \sin\left(\frac{t}{12}\right) + 0.8t$$

Ecuación 9: Función Trascendental que Representa el Tráfico de Red en Función al Tiempo en Horas.

donde:

- t representa el tiempo en horas.
- $f(t)$ representa la tasa de transferencia de datos en Mbps.

El análisis se realizará en el intervalo: $0 \leq t \leq 12$ correspondiente a una jornada operativa de 12 horas.

Para resolver este problema se emplearán los métodos de integración numérica del Trapecio y Simpson, permitiendo comparar precisión, estabilidad y comportamiento computacional.

6.2. Implementación del Método Numérico

6.2.1. Modelo Matemático

La integración numérica permite aproximar el área utilizando segmentos lineales:

$$\int_a^b f(x)dx \approx \frac{h}{2} [f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)]$$

Ecuación 10: Método de Trapecio para Aproximar Áreas dada una Función de X

Por otra parte, el método de Simpson utiliza aproximaciones parabólicas para obtener mayor precisión en funciones suaves:

$$\int_a^b f(x)dx \approx \frac{h}{3} [f(x_0) + 4 \sum f(x_{\text{impares}}) + 2 \sum f(x_{\text{pares}}) + f(x_n)]$$

Ecuación 11: Método de Simpson para Aproximar Áreas dada una Función de X con Número de Intervalos par

Para este proyecto se emplearon los siguientes parámetros:

- Intervalo de integración: $[0, 12]$
- Número de subintervalos:
 - Trapecio: $n = 6, 12, 24$
 - Simpson: $n = 6, 12, 24$

6.2.2. Algoritmo Implementado

6.2.2.1. Método del Trapecio

- Definir el intervalo de integración.
- Dividir el intervalos en n subintervalo.
- Evaluar la función en cada punto.
- Calcular la suma de áreas trapezoidales.
- Obtener la aproximación de la integral.

6.2.2.2. Método de Simpson

- Definir el intervalo de integración.
- Dividir el intervalo en un número par de subintervalos.
- Evaluar la función en cada punto.
- Aplicar ponderaciones pares e impares.
- Calcular la aproximación de la integral.

6.2.3. Implementación en MATLAB

6.2.3.1. Método del Trapecio

```
function [approximateIntegral, subintervalWidth, metadata] =
trapezoidal_rule( ...
    functionHandle, lowerBound, upperBound, subintervalCount)
narginchk(4, 4);

if ~isa(functionHandle, 'function_handle')
    error('trapezoidal_rule:InvalidFunctionHandle', ...
        'functionHandle must be a valid function handle.');
```

end

```
validateattributes(lowerBound, {'numeric'}, ...
    {'real', 'finite', 'scalar'}, mfilename, 'lowerBound',
2);
validateattributes(upperBound, {'numeric'}, ...
    {'real', 'finite', 'scalar'}, mfilename, 'upperBound',
3);
validateattributes(subintervalCount, {'numeric'}, ...
    {'real', 'finite', 'scalar', 'integer', 'positive'}, ...
    mfilename, 'subintervalCount', 4);

if upperBound <= lowerBound
    error('trapezoidal_rule:InvalidInterval', ...
        'upperBound must be greater than lowerBound.');
```



```

end

subintervalWidth = (upperBound - lowerBound) /
subintervalCount;
xNodes = linspace(lowerBound, upperBound, subintervalCount +
1);
yNodes = functionHandle(xNodes);

validateattributes(yNodes, {'numeric'}, ...
    {'real', 'finite', 'vector', 'numel', subintervalCount +
1}, ...
    mfilename, 'functionHandle(xNodes)');

yNodes = yNodes(:).';
approximateIntegral = subintervalWidth * ...
    ((yNodes(1) + yNodes(end)) / 2 + sum(yNodes(2:end-1)));

metadata = struct( ...
    'lowerBound', lowerBound, ...
    'upperBound', upperBound, ...
    'subintervalCount', subintervalCount, ...
    'subintervalWidth', subintervalWidth, ...
    'xNodes', xNodes, ...
    'yNodes', yNodes);
end

```

6.2.3.2. Método de Simpson

```

function [approximateIntegral, subintervalWidth, metadata] =
simpson_rule( ...
    functionHandle, lowerBound, upperBound, subintervalCount)
narginchk(4, 4);

if ~isa(functionHandle, 'function_handle')
    error('simpson_rule:InvalidFunctionHandle', ...
        'functionHandle must be a valid function handle.');
```

end



```

validateattributes(lowerBound, {'numeric'}, ...
    {'real', 'finite', 'scalar'}, mfilename, 'lowerBound',
2);
validateattributes(upperBound, {'numeric'}, ...
    {'real', 'finite', 'scalar'}, mfilename, 'upperBound',
3);
validateattributes(subintervalCount, {'numeric'}, ...
    {'real', 'finite', 'scalar', 'integer', 'positive'}, ...
    mfilename, 'subintervalCount', 4);

if upperBound <= lowerBound
    error('simpson_rule:InvalidInterval', ...
        'upperBound must be greater than lowerBound.');
```

end

```

if mod(subintervalCount, 2) ~= 0
    error('simpson_rule:InvalidSubintervalCount', ...
        'subintervalCount must be even for Simpson's
rule.');
```

end

```

    subintervalWidth = (upperBound - lowerBound) /
subintervalCount;
    xNodes = linspace(lowerBound, upperBound, subintervalCount +
1);
    yNodes = functionHandle(xNodes);

    validateattributes(yNodes, {'numeric'}, ...
        {'real', 'finite', 'vector', 'numel', subintervalCount +
1}, ...
        mfilename, 'functionHandle(xNodes)');
```

```

yNodes = yNodes(:).';
oddIndexedTerms = sum(yNodes(2:2:end-1));
evenIndexedTerms = sum(yNodes(3:2:end-2));
```

```

approximateIntegral = (subintervalWidth / 3) * ...
    (yNodes(1) + yNodes(end) + 4 * oddIndexedTerms + 2 *
evenIndexedTerms);

metadata = struct( ...
    'lowerBound', lowerBound, ...
    'upperBound', upperBound, ...
    'subintervalCount', subintervalCount, ...
    'subintervalWidth', subintervalWidth, ...
    'xNodes', xNodes, ...
    'yNodes', yNodes);
end

```

6.3. Validación del Resultado

La validación se realizó comparando los resultados obtenidos mediante los métodos del Trapecio y Simpson utilizando diferentes números de subintervalos.

Se analizaron:

- valor aproximado de la integral,
- influencia del número de subintervalos,
- precisión de los métodos,
- y comportamiento computacional.

Las gráficas y resultados numéricos generados en MATLAB permitieron observar que:

- El aumento en el número de subintervalos mejora la precisión de la aproximación.
- El método de Simpson produce resultados más precisos para funciones continuas y suaves.
- Ambos métodos convergen hacia valores similares a medida que aumenta el número de particiones.

Además, el archivo *summary_report.txt* almacena los resultados numéricos, errores y aproximaciones obtenidas durante las simulaciones.

6.4. Análisis y Conclusiones

La implementación de los métodos de integración numérica permitió estimar adecuadamente la cantidad total de datos transferidos en la red durante el intervalo analizado.

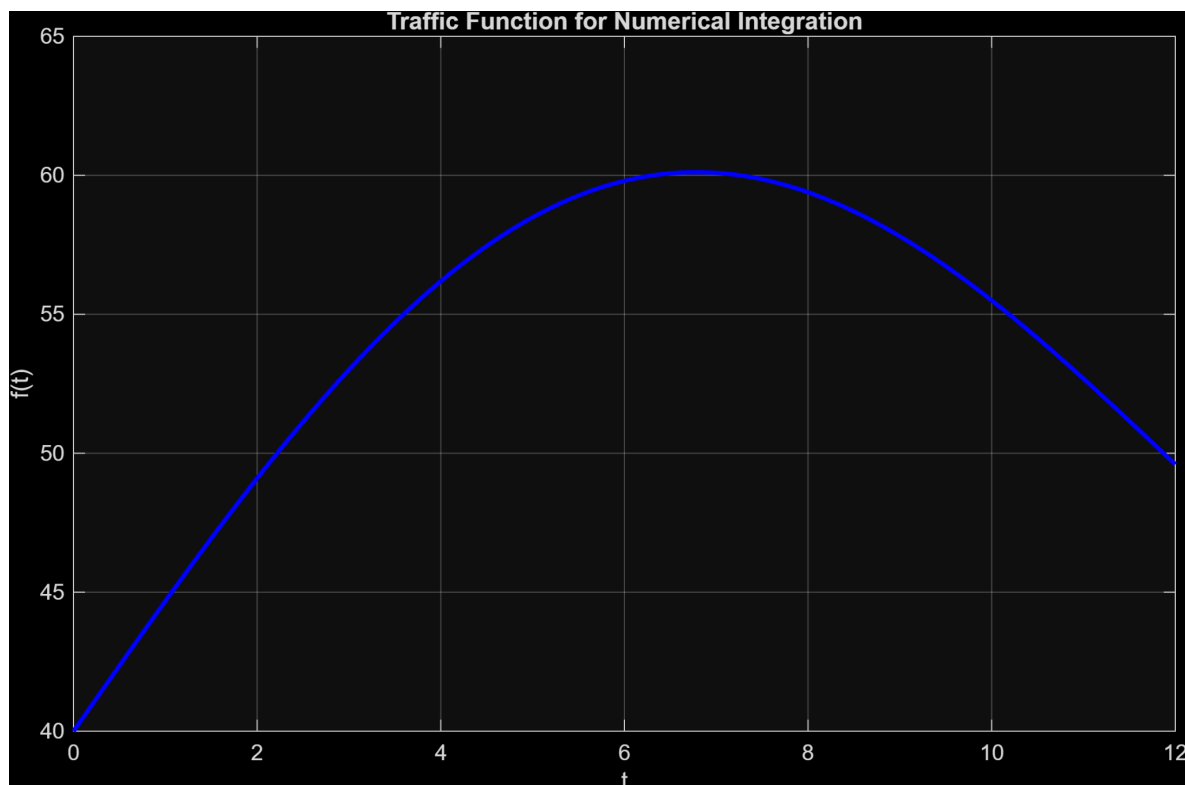
Los resultados evidenciaron que:

- El método de Simpson proporciona mayor precisión utilizando el mismo número de subintervalos.
- El método del Trapecio presenta una implementación más sencilla y eficiente.
- El aumento del número de particiones reduce el error de aproximación en ambos métodos.

Desde el punto de vista computacional, la integración numérica constituye una herramienta fundamental para estimar acumulaciones y consumos en sistemas donde no se dispone de soluciones analíticas exactas.

Finalmente, se concluye que los métodos del Trapecio y Simpson son técnicas eficientes para el análisis de tráfico y transferencia de datos en entornos computacionales, permitiendo realizar estimaciones confiables a partir de funciones o datos discretos.

Método	Subintervalos	Tamaño del Paso	Valor Aproximado	Error Absoluto
Trapecio	6	2	649.561524227066	2.63003479909833
Simpson	6	2	652.241016151378	0.049457125212939
Trapecio	12	1	651.536311690877	0.655247335287413
Simpson	12	1	652.194574178814	0.00301515264959562
Trapecio	24	0.5	652.027887661991	0.163671364173297
Simpson	24	0.5	652.19174631903	0.000187292864893607



6.5. Interpretación Técnica de Resultados

Los resultados evidencian que ambos métodos convergen al aumentar el número de subintervalos, pero Simpson presenta una ventaja significativa en precisión. Con 24 subintervalos, el error del método del Trapecio fue 0.163671, mientras que Simpson obtuvo un error de apenas 0.000187, lo que representa una precisión aproximadamente 874 veces mayor. Esto se explica porque la función de tráfico analizada es continua y suave, condición en la cual Simpson aprovecha mejor la aproximación parabólica. Por tanto, para estimaciones acumuladas de transferencia de datos en jornadas operativas, Simpson sería el método recomendado cuando se requiere alta precisión; Trapecio sería aceptable cuando se prioriza simplicidad de implementación.

7. Solución Numérica de EDOs

7.1. Descripción del Problema

En plataformas virtuales universitarias, la cantidad de usuarios conectados puede variar rápidamente durante ciertos periodos del día, especialmente en horarios de clases, evaluaciones en línea o procesos administrativos. Comprender la evolución temporal de usuarios activos permite planificar capacidad de servidores, ancho de banda y recursos computacionales.

Debido a que este comportamiento cambia continuamente en el tiempo, puede modelarse mediante una ecuación diferencial ordinaria que relacione la tasa de crecimiento de usuarios con la cantidad actual de conexiones.

Para abordar este problema, se propone el siguiente modelo logístico de crecimiento de usuarios:

$$\frac{dy}{dt} = 0.6y\left(1 - \frac{y}{500}\right)$$

Ecuación 12: Ecuación Diferencial Ordinaria que Representa un Modelo Logístico de Crecimiento de Usuarios en Función al Tiempo en Horas.

donde:

- $y(t)$ representa la cantidad de usuarios conectados.
- t representa el tiempo en horas.
- 500 corresponde a la capacidad máxima estimada del sistema.

La condición inicial utilizada es: $y(0) = 50$ lo que indica que inicialmente existen 50 usuarios conectados a la plataforma virtual.

Para resolver este problema se emplearán los métodos numéricos de Euler Lineal y Runge-Kutta de cuarto orden, permitiendo comparar precisión, estabilidad y comportamiento computacional.

7.2. Implementación del Método Numérico

7.2.1. Modelo Matemático

Las ecuaciones diferenciales ordinarias permiten modelar fenómenos dinámicos donde una variable cambia continuamente respecto al tiempo.

El método de Euler aproxima la solución utilizando la pendiente en el punto actual:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Ecuación 13: Ecuación del Método de Euler Lineal para Encontrar el n-ésimo más uno Termina Solución

Por otra parte, el método de Runge-Kutta de cuarto orden mejora la precisión utilizando evaluaciones intermedias:

$$y_{n+1} = y_n + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

Ecuación 14: Ecuación del Método de Runge-Kutta 4 para Encontrar el n-ésimo más uno Termina Solución

donde:

- $k_1 = hf(t_n, y_n)$



- $k_2 = hf(t_n + \frac{h}{2}, y_n + \frac{k_1}{2})$
- $k_3 = hf(t_n + \frac{h}{2}, y_n + \frac{k_2}{2})$
- $k_4 = hf(t_n + h, y_n + k_3)$

Para este proyecto se emplearon los siguientes parámetros:

- Intervalo de simulación: $0 \leq t \leq 10$
- Tamaños de paso:
 - $h = 1$
 - $h = 0.5$
 - $h = 0.25$

7.2.2. Algoritmo Implementado

7.2.2.1. Método de Euler Lineal

- Definir la ecuación diferencial y la condición inicial.
- Seleccionar el tamaño de paso.
- Calcular la pendiente en cada iteración.
- Actualizar el valor aproximado de la solución.
- Repetir hasta completar el intervalo de simulación.

7.2.2.2. Método de Runge Kutta de cuarto orden

- Definir la ecuación diferencial y la condición inicial.
- Seleccionar el tamaño de paso.
- Calcular los coeficientes k_1, k_2, k_3 y k_4
- Actualizar el valor aproximado de la solución.
- Repetir el procedimiento hasta completar el intervalo.

7.2.3. Implementación en MATLAB

7.2.3.1. Método de Euler Lineal

```
function [tValues, yValues, metadata] = euler_method( ...
    odeFunction, interval, initialValue, stepSize)
    narginchk(4, 4);

    if ~isa(odeFunction, 'function_handle')
        error('euler_method:InvalidFunctionHandle', ...
            'odeFunction must be a valid function handle.');
```

```
end

validateattributes(interval, {'numeric'}, ...
```



```

        {'real', 'finite', 'vector', 'numel', 2}, mfilename,
        'interval', 2);
        validateattributes(initialValue, {'numeric'}, ...
            {'real', 'finite', 'scalar'}, mfilename, 'initialValue',
        3);
        validateattributes(stepSize, {'numeric'}, ...
            {'real', 'finite', 'scalar', 'positive'}, mfilename,
        'stepSize', 4);

        interval = interval(:).';
        startPoint = interval(1);
        endPoint = interval(2);

        if endPoint <= startPoint
            error('euler_method:InvalidInterval', ...
                'interval(2) must be greater than interval(1).');
        end

        stepCount = round((endPoint - startPoint) / stepSize);
        reconstructedEndPoint = startPoint + stepCount * stepSize;

        if abs(reconstructedEndPoint - endPoint) > 1e-10
            error('euler_method:IncompatibleStepSize', ...
                'stepSize must divide the interval length exactly.');
```

```

        end

        tValues = linspace(startPoint, endPoint, stepCount + 1).';
        yValues = zeros(stepCount + 1, 1);
        slopeHistory = zeros(stepCount, 1);
        yValues(1) = initialValue;

        for stepIndex = 1:stepCount
            currentSlope = odeFunction(tValues(stepIndex),
            yValues(stepIndex));

            validateattributes(currentSlope, {'numeric'}, ...

```



```

        {'real', 'finite', 'scalar'}, mfilename,
        'odeFunction(t, y)');

        slopeHistory(stepIndex) = currentSlope;
        yValues(stepIndex + 1) = yValues(stepIndex) + stepSize *
currentSlope;
    end

    metadata = struct( ...
        'interval', interval, ...
        'initialValue', initialValue, ...
        'stepSize', stepSize, ...
        'stepCount', stepCount, ...
        'slopeHistory', slopeHistory);
end

```

7.2.3.2. Método de Runge Kutta de cuarto orden

```

function [tValues, yValues, metadata] = runge_kutta_4( ...
    odeFunction, interval, initialValue, stepSize)
    narginchk(4, 4);

    if ~isa(odeFunction, 'function_handle')
        error('runge_kutta_4:InvalidFunctionHandle', ...
            'odeFunction must be a valid function handle.');
```

```

    end

    validateattributes(interval, {'numeric'}, ...
        {'real', 'finite', 'vector', 'numel', 2}, mfilename,
        'interval', 2);
    validateattributes(initialValue, {'numeric'}, ...
        {'real', 'finite', 'scalar'}, mfilename, 'initialValue',
    3);
    validateattributes(stepSize, {'numeric'}, ...
        {'real', 'finite', 'scalar', 'positive'}, mfilename,
        'stepSize', 4);

```



```

interval = interval(:).';
startPoint = interval(1);
endPoint = interval(2);

if endPoint <= startPoint
    error('runge_kutta_4:InvalidInterval', ...
        'interval(2) must be greater than interval(1).');
end

stepCount = round((endPoint - startPoint) / stepSize);
reconstructedEndPoint = startPoint + stepCount * stepSize;

if abs(reconstructedEndPoint - endPoint) > 1e-10
    error('runge_kutta_4:IncompatibleStepSize', ...
        'stepSize must divide the interval length exactly.');
```

```

end

tValues = linspace(startPoint, endPoint, stepCount + 1).';
yValues = zeros(stepCount + 1, 1);
k1History = zeros(stepCount, 1);
k2History = zeros(stepCount, 1);
k3History = zeros(stepCount, 1);
k4History = zeros(stepCount, 1);
yValues(1) = initialValue;

for stepIndex = 1:stepCount
    currentTime = tValues(stepIndex);
    currentValue = yValues(stepIndex);

    k1 = odeFunction(currentTime, currentValue);
    k2 = odeFunction(currentTime + stepSize / 2, currentValue
+ stepSize * k1 / 2);
    k3 = odeFunction(currentTime + stepSize / 2, currentValue
+ stepSize * k2 / 2);
    k4 = odeFunction(currentTime + stepSize, currentValue +
stepSize * k3);

```



```

        validateattributes(k1, {'numeric'}, {'real', 'finite',
'scalar'}, mfilename, 'k1');
        validateattributes(k2, {'numeric'}, {'real', 'finite',
'scalar'}, mfilename, 'k2');
        validateattributes(k3, {'numeric'}, {'real', 'finite',
'scalar'}, mfilename, 'k3');
        validateattributes(k4, {'numeric'}, {'real', 'finite',
'scalar'}, mfilename, 'k4');

        k1History(stepIndex) = k1;
        k2History(stepIndex) = k2;
        k3History(stepIndex) = k3;
        k4History(stepIndex) = k4;

        yValues(stepIndex + 1) = currentValue + ...
            (stepSize / 6) * (k1 + 2 * k2 + 2 * k3 + k4);
    end

    metadata = struct( ...
        'interval', interval, ...
        'initialValue', initialValue, ...
        'stepSize', stepSize, ...
        'stepCount', stepCount, ...
        'k1History', k1History, ...
        'k2History', k2History, ...
        'k3History', k3History, ...
        'k4History', k4History);
end

```

7.3. Validación del resultado

La validación se realizó comparando las soluciones aproximadas obtenidas mediante los métodos de Euler y Runge-Kutta utilizando distintos tamaños de paso.

Se analizaron:

- aproximación obtenida en cada instante,

- sensibilidad al tamaño de paso,
- estabilidad numérica,
- y tiempo de ejecución computacional.

Las gráficas y resultados generados en MATLAB permitieron observar que:

- El número de usuarios presenta crecimiento acelerado inicial y posterior estabilización.
- El método de Runge-Kutta ofrece aproximaciones más precisas y suaves.
- La precisión de ambos métodos mejora al disminuir el tamaño de paso.
- Euler presenta mayor error acumulado para tamaños de paso grandes.

Además, el archivo ***summary_report.txt*** almacena resultados numéricos, aproximaciones y errores obtenidos durante las simulaciones.

7.4. Análisis y conclusiones

La implementación de los métodos numéricos para EDOs permitió modelar adecuadamente la evolución temporal de usuarios conectados en una plataforma virtual universitaria.

Los resultados evidenciaron que:

- El modelo logístico representa correctamente el crecimiento limitado por capacidad máxima.
- El método de Euler ofrece una implementación sencilla, aunque con menor precisión.
- El método de Runge-Kutta de cuarto orden proporciona resultados más estables y precisos.
- La reducción del tamaño de paso disminuye el error numérico en ambos métodos.

Desde el punto de vista computacional, los métodos de solución numérica de EDOs permiten analizar sistemas dinámicos y prever comportamientos futuros en entornos tecnológicos.

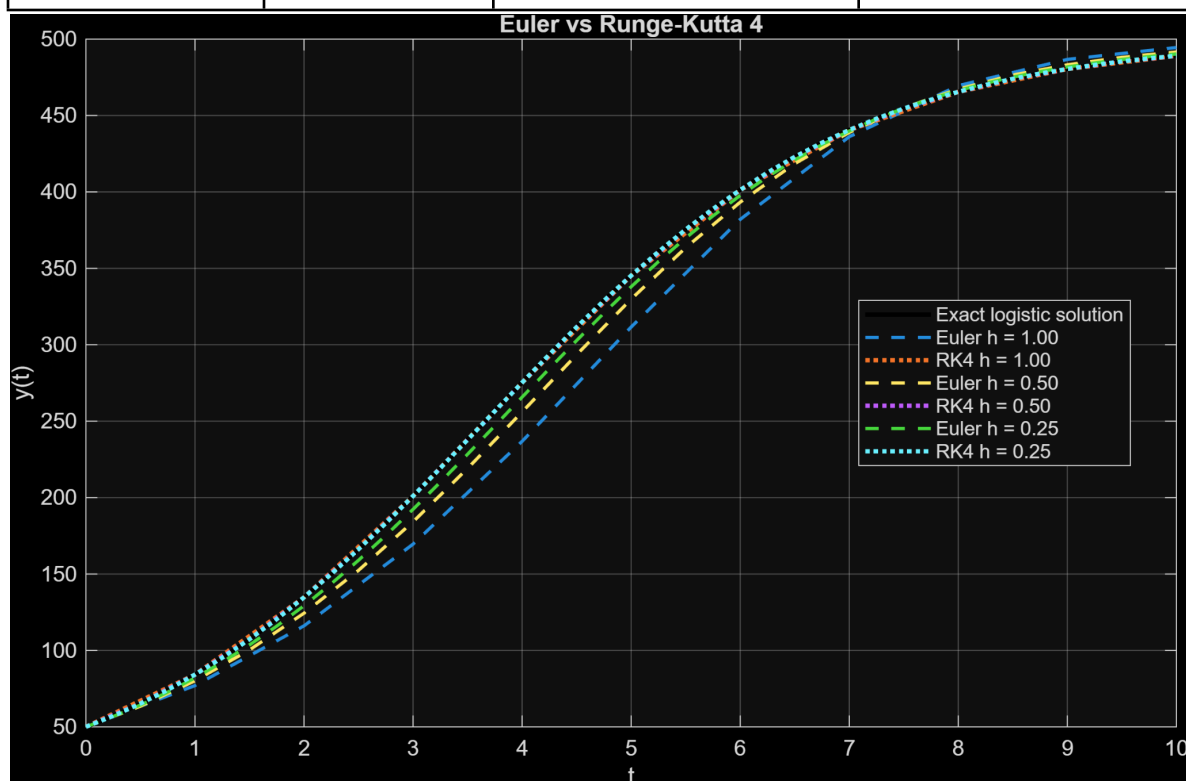
Finalmente, se concluye que los métodos de Euler y Runge-Kutta constituyen herramientas fundamentales para la simulación y análisis de fenómenos dinámicos en ingeniería de sistemas y administración de plataformas virtuales.

Método	Tamaño del Paso	Aproximación Final	Error Absoluto Máximo
--------	-----------------	--------------------	-----------------------





Euler	1	494.47101397456	38.4587123513292
Runge-Kutta 4	1	489.052299519907	0.0630613379786382
Euler	0.5	491.563451676153	19.2417383409824
Runge-Kutta 4	0.5	489.087046580992	0.00444978141410957
Euler	0.25	490.281029786944	9.61547714185167
Runge-Kutta 4	0.25	489.088909882683	0.000295959825734826



7.5. Interpretación Técnica de Resultados

La solución del modelo logístico muestra que la cantidad de usuarios tiende a estabilizarse cerca de la capacidad máxima de 500 usuarios, lo cual es coherente con el comportamiento esperado de un sistema limitado por recursos. La comparación entre métodos evidencia que Euler reduce su error aproximadamente a la mitad cada vez que se reduce el paso, comportamiento propio de un método de primer orden. En contraste, Runge-Kutta de cuarto orden disminuye el error de forma mucho más acelerada, alcanzando un error máximo de solo 0.000295 con $h = 0.25$. Por esta razón, RK4 resulta más adecuado para simular crecimiento de usuarios cuando se requiere precisión, mientras que Euler puede utilizarse únicamente como aproximación inicial o con fines educativos.

8. Conclusiones Generales

- 8.1. El desarrollo de este proyecto permitió evidenciar la importancia de los métodos numéricos como herramientas fundamentales para el análisis, modelado y optimización de sistemas informáticos universitarios. A través de la implementación en MATLAB de técnicas como Series de Taylor, métodos de búsqueda de raíces, interpolación polinómica, integración numérica y solución de ecuaciones diferenciales ordinarias, fue posible estudiar diferentes comportamientos asociados al rendimiento de un servidor web, tales como el uso de CPU, el tráfico de red, los tiempos de respuesta y la evolución de usuarios en el sistema.
- 8.2. Los resultados obtenidos demostraron que cada método numérico posee ventajas específicas según el tipo de problema abordado. Las Series de Taylor permitieron aproximar el comportamiento local de funciones dinámicas; los métodos de Bisección y Newton-Raphson facilitaron la identificación de puntos críticos de operación; la interpolación polinómica permitió estimar valores intermedios a partir de datos conocidos; la integración numérica hizo posible calcular acumulaciones de carga o tráfico; y los métodos de Euler y Runge-Kutta permitieron simular la evolución temporal de variables del sistema.
- 8.3. Asimismo, se comprobó que MATLAB constituye una herramienta adecuada para automatizar cálculos, validar resultados, generar gráficas y comparar el desempeño de distintos métodos numéricos. Esto permitió no solo obtener soluciones aproximadas, sino también interpretar técnicamente los resultados y relacionarlos con situaciones reales de administración de servidores y plataformas digitales académicas.

9. Conclusión Final del Documento

En conclusión, el desarrollo de este trabajo permitió cumplir satisfactoriamente el objetivo general planteado, al aplicar e implementar diversos métodos numéricos en MATLAB para analizar el comportamiento de un servidor web universitario. A través de este proceso, fue posible evidenciar cómo las herramientas matemáticas y computacionales pueden emplearse para representar, aproximar y estudiar fenómenos propios de los sistemas informáticos, tales como el uso del CPU, el tráfico de red, los tiempos de respuesta, la carga acumulada y la evolución de usuarios en un entorno digital académico.

La aplicación de métodos como las Series de Taylor, la búsqueda de raíces, la interpolación polinómica, la integración numérica y la solución de ecuaciones diferenciales ordinarias permitió abordar diferentes aspectos del rendimiento del servidor desde una perspectiva técnica y analítica. Cada método aportó una forma particular de interpretar el comportamiento del sistema: algunos facilitaron la aproximación de funciones, otros

permitieron identificar puntos críticos de operación, estimar valores faltantes, calcular acumulaciones de carga o simular la evolución temporal de variables relevantes.

Asimismo, los resultados obtenidos demostraron que los métodos numéricos no solo son herramientas teóricas, sino que también tienen una aplicación práctica importante en la ingeniería de sistemas y computación. Su uso permite generar información útil para la toma de decisiones técnicas, especialmente en escenarios donde se requiere anticipar fallas, prevenir saturaciones, optimizar recursos y garantizar una mejor prestación de los servicios digitales universitarios.

De igual manera, la implementación en MATLAB facilitó la automatización de los cálculos, la comparación entre métodos, la validación de resultados y la representación gráfica de los datos analizados. Esto permitió comprender con mayor claridad el comportamiento del sistema y establecer criterios técnicos para evaluar su desempeño bajo diferentes condiciones de demanda.

Finalmente, se puede afirmar que los métodos numéricos constituyen una herramienta fundamental para el análisis y optimización de sistemas informáticos, ya que contribuyen a anticipar escenarios de saturación, mejorar la planificación de los recursos computacionales y fortalecer la capacidad de respuesta de las plataformas digitales frente a condiciones de alta concurrencia. Por tanto, este trabajo no solo permitió aplicar los conocimientos adquiridos en el curso, sino también reconocer la importancia de estos métodos en la solución de problemas reales dentro del campo de la ingeniería.

10. Referencias bibliográficas

Burden, R. L., & Faires, J. D. (2011). *Análisis numérico* (9.^a ed.). Cengage Learning.

Chapra, S. C., & Canale, R. P. (1987). *Métodos numéricos para ingenieros: con aplicaciones en computadoras personales*. Dialnet (Universidad de la Rioja).

<https://dialnet.unirioja.es/servlet/libro?codigo=370874>

Chapra, S. C., & Canale, R. P. (2015). *Métodos numéricos para ingenieros* (7.^a ed.). McGraw-Hill Education.

Gerald, C. F., & Wheatley, P. O. (2004). *Applied numerical analysis* (7th ed.). Pearson Education.

Kincaid, D., & Cheney, W. (2009). *Análisis numérico: Las matemáticas del cálculo científico* (3.^a ed.). American Mathematical Society.

Kreyszig, E. (2011). *Matemáticas avanzadas para ingeniería* (10.^a ed.). Wiley.

Mathews, J. H., & Fink, K. D. (2004). *Métodos numéricos con MATLAB* (3.^a ed.). Pearson Educación.

Nakamura, S. (1997). *Métodos numéricos aplicados con software*. Prentice Hall.

[MATLAB Documentation](#)

Romero Cuero, E., et al. (2019). *Métodos numéricos con MATLAB para ingenieros*. Universidad del Quindío.



UNIQUEINDÍO, en conexión territorial

Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia

www.uniquindio.edu.co